

Chapter 11: Introduction to MongoDB

Gregory S. DeLozier, PhD

Sometimes tables are constraining. We have been translating application objects into rows, but not every object has the same fields, and some objects contain smaller objects that we usually want to retrieve together. A document database lets us store named fields along with lists and nested documents. (1)

When the Fields Vary

Imagine a database of medical test results. A blood chemistry test and an imaging examination record different kinds of information. A single table with a column for every possible result would have many columns unused by most records.

Here is a small, invented example. A dash means the field does not apply to that test.

Test	Glucose (mg/dL)	Sodium (mmol/L)	Body region	Report
Blood chemistry	92	140	-	-
X-ray	-	-	Left wrist	Report text

We could use separate relational tables for different test types. With documents, another option is to keep the common fields and let the result fields vary:

```
{"test": "Blood chemistry", "patient_id": "P17",  
  "results": {"glucose_mg_dL": 92, "sodium_mmol_L": 140}}  
  
{"test": "X-ray", "patient_id": "P17",  
  "results": {"body_region": "Left wrist", "report": "Report text"}}
```

The values are just examples of stored data. Each record contains the fields appropriate to that test. We still need clear names, units, and rules for interpreting the values. A field that is absent is not a measurement of zero.

When We Read the Parts Together

Consider one grocery purchase. We normally want the purchase and its line items together: how much of each product was bought, and the price paid. A stock keeping unit (SKU) identifies a product in the store's catalog.

In a relational design, a `purchase_item` table connects purchases and products. A purchase has many products, and a product appears in many purchases. The connecting rows also hold the quantity and sale price. Retrieving the receipt means bringing the purchase and its line items back together; product details may require another join.

A purchase document can contain its line items:

```
purchase = {
  "purchase_id": "R1042",
  "items": [
    {"sku": "MILK-1L", "quantity": 2, "unit_price_cents": 249},
    {"sku": "BREAD-W", "quantity": 1, "unit_price_cents": 325},
  ],
}
```

Reading this document brings back both line items. The product catalog can remain separate, with each SKU referring to its product. The price here is the price paid during this purchase, so a later catalog price change does not rewrite the receipt.

Embedding related data can avoid repeated joins when the application usually reads the parts together. That can reduce the work needed to retrieve a purchase. Indexed joins can also be fast; the performance difference depends on the data and the queries. MongoDB's modeling guidance recommends considering which values are read together when choosing between embedding and references. (2)

The purchase owns its line items. The product catalog serves many purchases. That distinction helps us decide what to embed and what to keep separate. A report across all purchases still has to examine the relevant line items, wherever we store them.

Where the Idea Came From

Document-oriented databases predate MongoDB. Lotus Notes, released in 1989, used document-oriented records for collaborative applications. Its forms and records were an earlier way to build applications around documents. (3)

Apache approved the establishment of its CouchDB project in November 2008. CouchDB stores documents using JavaScript Object Notation (JSON) and provides its own interface and replication system. It is a separate document-database tradition, not a MongoDB implementation. (4) (5)

10gen, the company behind MongoDB, was founded in 2007 and launched MongoDB in 2009. The original work was part of a broader cloud computing stack; the company then concentrated on the database. (6) (7)

Milestone	What it illustrates
Lotus Notes, 1989	Documents as application records
Apache CouchDB project, 2008	Another document database with its own interface
MongoDB launch, 2009	Documents in a server aimed at application developers

MongoDB became a common reference point for document databases. One sign of that influence is the appearance of other systems offering compatibility with its application programming interface (API). Amazon DocumentDB is a separate service with MongoDB API compatibility and documented functional differences. Familiar calls do not guarantee identical behavior. (8) (9)

Mongita gives us a smaller local version of this programming interface: Python methods for working with documents, without a database server. It implements a subset of the MongoDB/PyMongo API. PyMongo is the Python driver for MongoDB. (10)

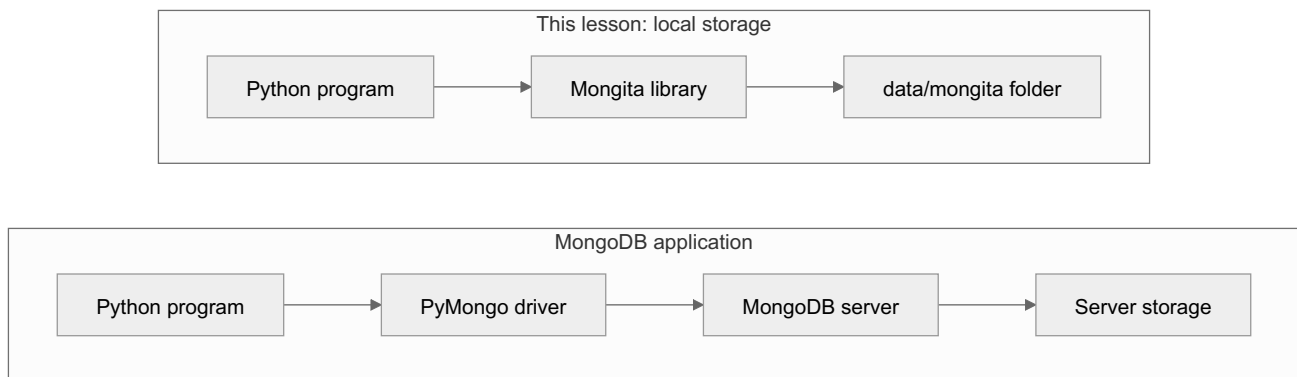
Begin with Mongita

We will begin with Mongita and the animal-record examples from our interactive Mongita session. Mongita runs inside Python and stores data locally. It implements a subset of the MongoDB/PyMongo API, so we can learn the basic operations without setting up a database server. PyMongo is the Python driver used to communicate with MongoDB. (10)

Where the Database Runs

With SQLite, the database engine runs inside our application. Mongita works that way too. In this chapter, `MongitaClientDisk` writes into a folder named `data/mongita` beside the example programs. We can exit Python and come back to the same records.

MongoDB uses a different arrangement: our Python program uses a client library to communicate with a database server. That server may be local or hosted elsewhere. Mongita lets us begin with familiar method calls, but it does not run a MongoDB server or implement every MongoDB feature. (10)



Both arrangements give us persistent data. The distinction here is where the database work runs. We will use one Python session at a time with the local lesson database.

Documents and Collections

Here are the two starting records:

```
{"name": "Meercat", "does_not_eat": "Snakes"}  
{"name": "Yellow mongoose", "eats": "Termites"}
```

Each dictionary represents a **document**. The keys name its fields. Notice that both documents have `name`, but only one has `eats`. We did not create a table with a fixed set of columns before storing these records.

A **collection** holds documents. Our database is named `hello_world_db`, and its collection is named `mongoose_collection`.

Relational term	Document database term	In this example
Table	Collection	<code>mongoose_collection</code>
Row	Document	One mongoose record
Column	Field	<code>name</code> , <code>eats</code> , or <code>weight</code>
Primary key	<code>_id</code> field	Identifier for one document

These are useful comparisons, not identical storage rules. A document can have fields that its neighbors do not have. Our application still needs to decide what a record means and which fields it needs. For example, an animal without an `eats` field has no recorded value for that field; that does not tell us that it eats nothing.

MongoDB stores documents using BSON (Binary JSON), a binary representation of JSON-like documents. JSON means JavaScript Object Notation; BSON extends the available types to include values such as `ObjectId`. In Python, we will work with dictionaries and the corresponding Python value types. (11)

Set Up the Examples

Use Python 3.10 or newer.

How to Read the Code in This Chapter

Try This marks commands to run. Other code blocks are reference examples or expected output. Run each interactive sequence in order in one Python session; later blocks use variables from earlier ones. To repeat the detailed walkthrough, exit Python and run `python setup_database.py --reset` in the introductory example's code directory, then start Python again.

Try This: From the course repository root, enter the example directory and install its dependencies in a virtual environment. The environment below lives in your home directory, outside the repository.

```
cd chapter-11-intro-to-mongo/code
python3 -m venv ~/venvs/advanced-database-mongita
source ~/venvs/advanced-database-mongita/bin/activate
python -m pip install -r requirements.txt
python setup_database.py
```

The setup program inserts the two starting documents when the collection is empty. Running it again leaves existing documents alone. **Try This (restart):** To start the lesson over, use:

```
python setup_database.py --reset
```

Reset deletes all documents in this lesson's `mongoose_collection` and inserts the two starting records again. It does not reset other collections. Use it before beginning the sequence below if you have already experimented with this collection.

The repository ignores `data/` folders. Keep generated database files there so that your local work does not become part of a Git commit.

Try This (complete demonstration): There is also a complete demonstration with printed results:

```
python mongita-session-clean.py --reset
```

That command resets the same collection and runs the whole sequence. For the hands-on version below, run `python setup_database.py --reset` afterward, then start Python:

Try This:

```
python
```

The remaining **Try This** blocks run in this same Python session unless a block says otherwise. Copy the code, not output lines. The setup has already inserted the two starting records.

A Short Interactive Example

Try This: After setup, start Python in the code directory. We will use `animals` as a short name for the existing collection. This adds one record and removes only that record at the end.

```
from lesson import connect
from bson.objectid import ObjectId

client, animals = connect()
result = animals.insert_one({"name": "Casey", "type": "dog"})
id_text = str(result.inserted_id)
print(id_text)

key = {"_id": ObjectId(id_text)}
print(animals.find_one(key))
animals.update_one(key, {"$set": {"age": 3}})
print(animals.find_one(key))
animals.delete_one(key)
print(animals.find_one(key)) # None
client.close()
```

The first read has no age field. The second has age equal to 3. The last read returns `None` because we deleted the document. The stored `_id` is an `ObjectId`; `id_text` is its 24-character hexadecimal string representation. This is the form we will pass through the web application. (12)

These four operations are enough to start replacing the SQL statements inside our database layer. Next we will use them in the Pets application. The longer interactive walkthrough follows the application section.

Replace the Pets Database Layer

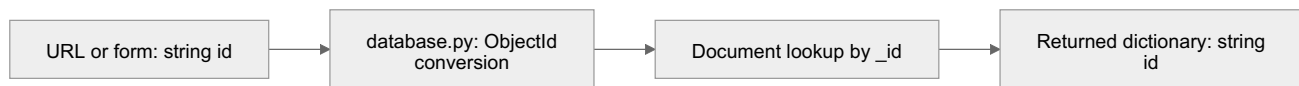
The code/pets-app folder contains the existing Flask Pets application with a Mongita database.py. It uses separate pets and owners collections. An owner is shared by several pets, so each pet refers to its owner rather than embedding another copy of the owner.

The routes still call create_pet(data), get_pet(id), update_pet(id, data), and delete_pet(id). Owner operations follow the same pattern. The implementation changes inside the layer.

Operation	Relational implementation	Mongita implementation
Create	Insert a row	insert_one(document)
Read one	Select by primary key	find_one({"_id": object_id})
Update	Update selected columns	update_one(filter, {"\$set": values})
Delete	Delete by primary key	delete_one({"_id": object_id})

IDs at the Application Boundary

The application keys are now string representations of ObjectIds, rather than integer row IDs. The documents still store ObjectId values. (12)



For example, the layer's creation function is:

```
def create_pet(data):
    pet = _normalize_pet_data(data)
    result = pets_collection.insert_one(pet)
    return str(result.inserted_id)
```

_normalize_pet_data() checks the required fields, converts the age to a non-negative integer, and looks up the owner. An omitted or blank age defaults to zero; negative or non-numeric input is rejected. The owner must exist. It saves owner_id as an ObjectId, too.

On a read, we convert the incoming string back before constructing the filter:

```
def get_pet(id):
    object_id = _to_object_id(id, "pet id")
    pet = pets_collection.find_one({"_id": object_id})
    if pet is None:
        return None
    return pet_to_dict(pet)
```

`_to_object_id()` uses `ObjectId(str(value))` and reports invalid identifiers as a `ValueError`.
`pet_to_dict()` turns `_id` into the application's `id` field and converts both identifiers to strings:

```
"id": str(pet["_id"]),  
"owner_id": str(pet["owner_id"]),
```

The other pet fields remain ordinary Python values. A template can still use `pet["id"]` in a link. Flask routes use `/update/<id>` so that the identifier arrives as text; an `<int: id>` converter would no longer fit.

Update and Delete

The existing implementation checks that the pet exists, then performs the document operation:

```
def update_pet(id, data):  
    object_id, _ = _require_existing_pet(id)  
    pet = _normalize_pet_data(data)  
    pets_collection.update_one({"_id": object_id}, {"$set": pet})  
  
def delete_pet(id):  
    object_id, _ = _require_existing_pet(id)  
    pets_collection.delete_one({"_id": object_id})
```

`$set` changes the supplied fields while keeping the document's identifier. The layer also checks owner relationships: a pet needs an existing owner, and an owner with pets cannot be deleted through these functions. These are application-layer checks in this example. They run when callers use these functions; they are not SQL foreign-key declarations.

Run the Application

Try This: Exit the introductory Python session with `exit()`. From `chapter-11-intro-to-mongo/code`, enter the application directory:

```
cd pets-app  
python -m pip install -r requirements.txt  
flask --app app run --debug
```

Use the Python environment created earlier. Open the address Flask prints, create an owner, and then create a pet. Follow its Update link and look at the identifier in the address. Change the pet, return to the list, and verify the saved values. Restart Flask and check that the records remain.

Storage is `data/mongita` beside the application's `database.py`. Setup opens the collections without resetting them. This starts a new document database; it does not convert the earlier SQLite file. Keep the generated `data/` folder out of Git.

Open the Collection

Try This: Stop Flask with Ctrl+C. From `chapter-11-intro-to-mongo/code/pets-app`, return to the introductory example's code directory:

```
cd ..
python setup_database.py --reset
python
```

This reset restores the two animal records for the sequence below. It does not touch the Pets application's separate database.

Try This:

```
from lesson import connect
client, mongoose_collection = connect()
print(mongoose_collection.count_documents({}))
```

Output:

```
2
```

The `connect()` helper keeps the location consistent between the setup program, the interactive session, and the complete demonstration. Its database code is short:

```
from pathlib import Path
from mongita import MongitaClientDisk

DATA_DIR = Path(__file__).resolve().parent / "data" / "mongita"

DATA_DIR.mkdir(parents=True, exist_ok=True)
client = MongitaClientDisk(host=str(DATA_DIR))
hello_world_db = client.hello_world_db
mongoose_collection = hello_world_db.mongoose_collection
```

This excerpt is from a Python file, where `__file__` names that file. In the interactive session, use the helper rather than retyping the excerpt. Selecting a database and collection gives us handles through which to work; we do not need a `CREATE TABLE` statement.

The seed operation in `lesson.py` is:

```
mongoose_collection.insert_many([
    {"name": "Meercat", "does_not_eat": "Snakes"},
    {"name": "Yellow mongoose", "eats": "Termites"},
])
```

Do not run that insertion again during the sequence. Each insertion creates more documents, even if their names match existing names. The name field is ordinary data, not a unique key. The example records also appear in Mongita’s introductory example. (10)

Retrieve a Document

Try This:

```
yellow = mongoose_collection.find_one({"eats": "Termites"})
print(yellow)
print(yellow["name"])
print(yellow["_id"])
```

`find_one()` returns a dictionary when it finds a match, or `None` when it does not. Use brackets to access a dictionary field: `yellow["name"]`. (13)

The dictionary includes the original fields and an automatically assigned `_id`. We did not supply that field when inserting the document. MongoDB normally uses an `ObjectId` for an automatically generated identifier, and Mongita does the same in this example. (11)

A filter such as `{"eats": "Termites"}` describes which documents we want. An empty filter, `{}`, places no conditions on the match.

Filter	Meaning in our example
<code>{}</code>	All documents
<code>{"name": "Meercat"}</code>	Documents with this name
<code>{"weight": {"\$gt": 1}}</code>	Documents whose numeric weight exceeds 1

These dictionaries play the role that a `WHERE` condition played in our Structured Query Language (SQL) queries. `$gt` means greater than. We will add `weight` next.

Update a Field

Try This:

```
result = mongoose_collection.update_one(
    {"name": "Meercat"},
    {"$set": {"weight": 2}},
)
print(result.matched_count, result.modified_count)
print(mongoose_collection.find_one({"name": "Meercat"}))
```

The counts are 1 1 on the first run after setup. The first dictionary selects the document. The second describes the change. `$set` adds `weight` when it is absent or assigns a new value when it is present. The other fields remain. The result separates how many documents matched from how many changed. (14)

We added a field to one document. The Yellow mongoose document still has no `weight`. A numeric query for weights greater than 1 now finds Meercat and does not find Yellow mongoose.

A Cursor Is a Consumable Result

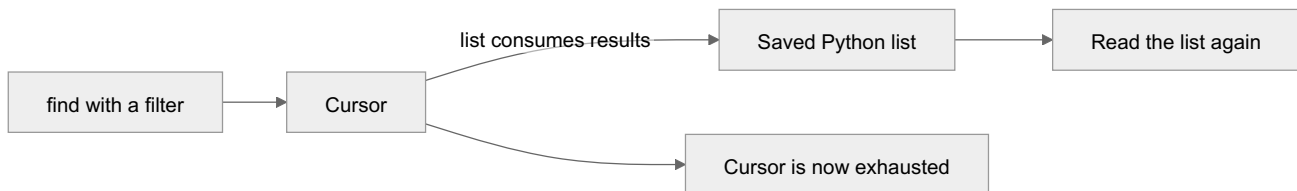
Try This:

```
cursor = mongoose_collection.find({"weight": {"$gt": 1}})
mongooses = list(cursor)
print(len(mongooses))
print(list(cursor))
print(len(mongooses))
```

Output:

```
1
[]
1
```

`find()` returns a **cursor**, which we can iterate to obtain matching documents. (13) The first `list(cursor)` consumes its results. Asking that same cursor for another list gets nothing because it has already reached the end. The saved `mongooses` list still contains the document. We can iterate that list again.



For a fresh query, call `find()` again. To keep a small result for repeated use, save it in a list. The list holds all those results in memory, so that choice matters when a query returns many documents.

Do not confuse this with `list(yellow)`. `yellow` is a dictionary, so iterating it produces its keys. It is already one document.

Try This:

```
print(list(yellow))
print(list(mongoose_collection.find({"weight": {"$gt": 300}})))
print(mongoose_collection.find_one({"weight": {"$gt": 300}}))
```

The first line prints field names. The second prints `[]`; the third prints `None`. Both queries found no matching document, but their return types differ.

Carry an Identifier as Text

A web address or form field carries text. Our document's `_id` is an `ObjectId`. We can convert it to text for transport and construct an `ObjectId` from that text when looking up the document. The constructor accepts a valid 24-character hexadecimal representation. (12)

Try This:

```
from bson.objectid import ObjectId

identifier = str(yellow["_id"])
print(identifier)
found = mongoose_collection.find_one({"_id": ObjectId(identifier)})
print(found["name"])
```

The name is Yellow mongoose. The identifier itself will differ from one setup to another. Keep the conversion explicit so that our lookup uses the same value type as the stored field.

Delete and Insert Again

Deleting a document removes that record. Inserting another document with the same name creates a new record. The old fields do not come back unless we supply them.

Try This:

```
old = mongoose_collection.find_one({"name": "Meercat"})
result = mongoose_collection.delete_one({"_id": old["_id"]})
print(result.deleted_count)
print(mongoose_collection.find_one({"_id": old["_id"]}))
```

Output is 1, then None. `delete_one()` removes one matching document and reports the count. (15) Using `_id` identifies the particular document we read.

Try This:

```
inserted = mongoose_collection.insert_one(
    {"name": "Meercat", "does_not_eat": "Snakes"}
)
print(inserted.inserted_id != old["_id"])
print(list(mongoose_collection.find({"weight": {"$gt": 1}})))
print(mongoose_collection.count_documents({}))
```

Output:

```
True
[]
2
```

The replacement has a new identifier and no `weight` field. There are two documents again, but the database is not in the same state it was in before the deletion.

Verify Persistence

Try This: Close this Python session:

```
client.close()
exit()
```

Try This: From the same code directory, start python again. Do not run setup or reset between sessions. Then run:

```
from lesson import connect
client, mongoose_collection = connect()
print(mongoose_collection.count_documents({}))
print(mongoose_collection.find_one({"name": "Meercat"}))
client.close()
```

The count is still 2. The Meercat document is the one we reinserted, without `weight`. The Python variables from our earlier session are gone; the saved documents remain in the configured folder.

We now have the operations needed to build a database layer: create, read, update, and delete, usually abbreviated CRUD. Our application can continue to call functions such as `get_pet()` or `update_pet()`. Inside those functions, document operations can take the place of SQL statements.

Exercise: Extend the Collection

Start with `python setup_database.py --reset`, then open the collection in Python. Add three invented animal records. Give two of them a numeric weight; leave it out of the third. Keep the names distinct so that you can follow each record.

Write a filter that returns exactly two of your new records. Save the result in a list and print their names. Then change one weight with `update_one()` and run the same query again. Explain why the membership changed or stayed the same.

Delete one of your records by `_id`. Exit Python and reopen the database to verify which documents remain. Keep your commands in a Python file so that someone else can follow what you did. Decide whether that file is intended to continue existing work or start over; make any reset explicit.

References

1. Wikipedia contributors. “Document-oriented database.” https://en.wikipedia.org/wiki/Document-oriented_database
2. MongoDB, Inc. “Best Practices for Data Modeling in MongoDB.” MongoDB Manual. <https://www.mongodb.com/docs/manual/data-modeling/best-practices/>

3. Wikipedia contributors. “HCL Notes.” History and database design. https://en.wikipedia.org/wiki/HCL_Notes
4. Apache Software Foundation. “Board history: CouchDB.” Resolution of November 19, 2008. <https://www.apache.org/foundation/board/minutes/CouchDB.html>
5. Apache CouchDB contributors. “Technical Overview.” <https://docs.couchdb.org/en/stable/intro/overview.html>
6. MongoDB, Inc. “Our Story.” <https://www.mongodb.com/company/our-story>
7. MongoDB, Inc. “10gen Announces Company Name Change to MongoDB, Inc.” August 27, 2013. <https://www.mongodb.com/company/newsroom/press-releases/10gen-announces-solutions-name-change-mongodb-inc>
8. Amazon Web Services. “Supported MongoDB APIs, operations, and data types in Amazon DocumentDB.” <https://docs.aws.amazon.com/documentdb/latest/devguide/mongo-apis.html>
9. Amazon Web Services. “Functional differences: Amazon DocumentDB and MongoDB.” <https://docs.aws.amazon.com/documentdb/latest/devguide/functional-differences.html>
10. Scott Rogowski and contributors. “Mongita.” Project README, introductory example, and API summary. <https://github.com/scottrogowski/mongita>
11. MongoDB, Inc. “Documents.” MongoDB Manual. <https://www.mongodb.com/docs/manual/core/document/>
12. MongoDB, Inc. “objectid - Tools for working with MongoDB ObjectIds.” PyMongo documentation. <https://pymongo.readthedocs.io/en/stable/api/bson/objectid.html>
13. MongoDB, Inc. “Find Documents.” PyMongo Driver documentation. <https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/query/find/>
14. MongoDB, Inc. “Update Documents.” PyMongo Driver documentation. <https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/update/>
15. MongoDB, Inc. “Delete Documents.” PyMongo Driver documentation. <https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/delete/>

Questions for Thought

1. In your current collection, which documents have no `weight` field? Inspect them. How would treating a missing weight as zero change the meaning of a report?
2. Save a query result as a list, update one of the matching documents, and run the query again. What does the saved list contain, and what does the new query return?
3. Insert two documents with the same name. How can you update exactly one of them? What should an application use as its identifier?
4. Where would you put a rule that every animal must have a name? How would that choice affect a second program that writes to the same database?
5. Which functions in our earlier database abstraction layer could keep their names and arguments when we replace relational storage with document storage? Which return values would need attention?
6. For a grocery receipt, which fields belong to the purchase and which belong to the product catalog? What should happen to an old receipt when the catalog price changes?

7. Compare retrieving one complete purchase with reporting total sales for one SKU across every purchase.
How might those two queries affect your choice to embed line items?